Day 10 — DeFi: an exchange with no company behind it

Constant-product AMM (x · y = k) · Sat 24 Oct 2026 · Phase: Smart contracts & DeFi · Learning Blockchain, day 10 of 15

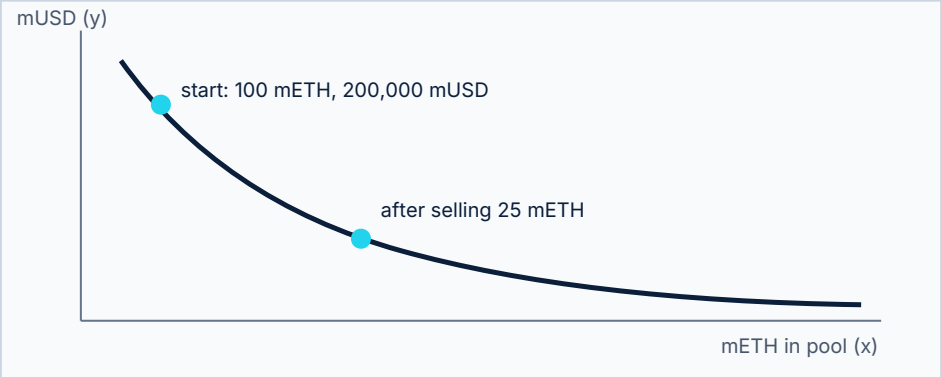
1. Learning objectives

- Explain how a swap pool prices trades using only `x · y = k`.
- Compute output, fee and price impact by hand and in code.
- Explain what liquidity providers earn and risk (fees vs impermanent loss).
- Describe a sandwich attack and why `minAmountOut` limits it.
- Name the safety rules of a pool: own books, count what arrived, locked shares, deadlines, re-entrancy lock.

2. The concept, step by step

The jar. A jar holds 100 apples and 100 oranges (100 × 100 = 10,000). Put in 25 apples: the jar has 125 apples, so it may hand out oranges only down to 10,000 ÷ 125 = 80. You get **20** oranges, not 25. The more you take, the dearer each next orange. Nobody sets the price; the jar does.

1. **Pool.** A contract holds two tokens (reserves `x`, `y`). Anyone may add both and get **LP shares**.
2. **Swap.** Trader sends `Δx`. After a 0.30% fee the contract pays out `Δy = Δx·0.997·y / (x + Δx·0.997)`, rounded **down**.
3. **Fee stays inside.** So `k` grows after each trade and every LP share is worth a bit more.
4. **Price = ratio.** 100 mETH vs 200,000 mUSD means 1 mETH ≈ 2,000 mUSD. Big trades move the ratio (price impact).
5. **Protection.** Every call carries a slippage limit and a deadline.



Every trade slides along the curve; it never reaches zero.

Sandwich attack

Step	Who	What happens
1	Victim	Sends a swap; it waits in the public mempool.
2	Bot	Sees it, buys first with a higher tip. Price moves against the victim.
3	Victim	Swap runs at the worse price.
4	Bot	Sells right after and keeps the difference.

Defence: `minAmountOut`. If the victim says "at least 99.5% of the quote", the bot can squeeze 0.5% at most; ask for more and the trade simply cancels.

3. Key code walkthrough

The price formula (rounds against the trader)

```
function getAmountOut(uint256 amountIn, uint256 reserveIn, uint256 reserveOut)
    public pure returns (uint256) {
  if (amountIn == 0) revert ZeroAmount();
  if (reserveIn == 0 || reserveOut == 0) revert InsufficientLiquidity();
  uint256 amountInWithFee = amountIn * (BPS - FEE_BPS);   // 0.997 as 9970/10000
  return (amountInWithFee * reserveOut)
       / (reserveIn * BPS + amountInWithFee);              // integer division = round DOWN
}
```

Count what really arrived

```
function _pull(IERC20 token, uint256 amount) private returns (uint256) {
  uint256 before = token.balanceOf(address(this));
  token.safeTransferFrom(msg.sender, address(this), amount);
  return token.balanceOf(address(this)) - before;   // fee-on-transfer safe
}
```

Own books, never `balanceOf` for prices

```
uint256 public reserve0;  uint256 public reserve1;   // changed only by add / remove / swap
// swap: received = _pull(...); out = getAmountOut(received, rIn, rOut);
//       if (out == 0 || out < minAmountOut) revert InsufficientOutput(out, minAmountOut);
//       _update(...);  tOut.safeTransfer(to, out);       // effects, then interaction
```

Evening improvement: ask the pool what a deposit will do

```
(uint256 shares, uint256 a0, uint256 a1) = pair.quoteAddLiquidity(d0, d1);
pair.addLiquidity(d0, d1, shares * 995 / 1000, me, deadline);   // minShares from the quote
```

4. Results from the run

Check	Result
Sell 1 / 5 / 50 mETH in a 100 / 200,000 pool	1,974.32 / 9,496.59 / 66,533.20 mUSD (impact 1.28% / 5.03% / 33.47%)
Fees: 10 round trips of 5 mETH	Bob loses 0.2854 mETH; the pool's `k` grows 0.29%; Alice withdraws more than she put in
Sandwich, no limit	Victim loses 29.97%, bot earns 1.4426 mETH
Sandwich, 0.5% limit	Victim loses 0.50%, bot earns 0.0245 mETH
Donation attack	Failed: victim got 10 mETH back
Impermanent loss, price ×2 (exercise 2)	Formula −5.72%, real contract −5.68%
Tests	33 / 33 pass; fuzz 150 programs, 860 steps, 4 invariants held

5. QA & security findings

ID	Sev.	Finding	Outcome
L1	Low	addLiquidity accepted the pool or a token as recipient (shares lost forever)	Fixed (morning)
T1	Low	My fuzz test could not tell expected reverts from bugs	Fixed (morning)
E1–E3	Info	Deadline boundary, emptied pool, pool opened at a silly price	Tested, no defect
E4 / K6	Info	A token that takes balance from the pool makes it insolvent for the last LP	Known issue, documented

No Low, Medium or High issues were found in the evening pass. ESLint, Solhint clean; `npm audit` 0 vulnerabilities; Slither results explained in QA.md.

6. Common mistakes & interview questions

- No `minAmountOut` (or set to 0): a bot decides your price.
- Reading `balanceOf` to price or share: a donation breaks it.
- Rounding in the user's favour: bots repeat it millions of times.
- Using the pool's spot price as an oracle for lending: a flash loan moves it for one block.

Q1. What does x · y = k mean?
After a swap the product of the two reserves must not decrease. It fixes the price curve.

Q2. Why is a big trade worse per token?
It moves along the curve; each extra token costs more (price impact), plus the 0.30% fee.

Q3. How do LPs earn, and what is impermanent loss?
Fees stay in the pool so shares grow. If the outside price moves, arbitrage rebalances the pool and the LP ends with less value than just holding; for a 2× move it is about 5.7%.

Q4. How does a sandwich work and how do you limit it?
A bot buys before and sells after your trade. Set `minAmountOut` (and a deadline); use private mempools for the rest.

Q5. Why lock 1,000 shares on the first deposit?
So the first depositor cannot make one share worth an absurd amount and round later depositors to zero (inflation attack).

7. Exercises summary

Level	Task	Run
Easy	Price-impact table for any pool	`npm run ex1`
Medium	Measure impermanent loss on the real contract and compare with 2√r/(1+r) − 1	`npm run ex2`
Challenge	Exact arbitrage trade size by integer bisection	`npm run ex3`

Cheat sheet
Out = Δx·0.997·y / (x + Δx·0.997) · fee 0.30% stays in pool · price = y / x
IL = 2√r / (1 + r) − 1 (r = price change) · first deposit shares = √(x·y) − 1,000
Always: `minAmountOut`, deadline, own books, count what arrived, round against the user, re-entrancy lock.
Never: use a pool spot price as an oracle; list rebasing tokens.

8. References

1. Uniswap V2 whitepaper — https://docs.uniswap.org/whitepaper.pdf
2. Uniswap V2 docs: how it works — https://docs.uniswap.org/contracts/v2/concepts/protocol-overview/how-uniswap-works
3. Uniswap V2 pair contract — https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Pair.sol
4. ethereum.org, Decentralized finance — https://ethereum.org/en/defi/
5. ethereum.org, Maximal extractable value (MEV) — https://ethereum.org/en/developers/docs/mev/
6. OpenZeppelin Contracts 5.x docs — https://docs.openzeppelin.com/contracts/5.x/
7. Solidity documentation — https://docs.soliditylang.org/
8. Smart Contract Weakness Classification registry — https://swcregistry.io/
9. Angeris et al., "An analysis of Uniswap markets" — https://arxiv.org/abs/1911.03380
10. Slither static analyzer — https://github.com/crytic/slither